Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

**Звіт**
**До лабораторної роботи №8**
**Варіант 14**
**На тему**
**«Алгебраїчні типи даних у Python»**

**Виконав:**
**Студент групи ФЕП-21с**
**Хороший Святослав**

**Перевірив:**
**Ас. Остріков Д. А.**

Львів 2026

**Мета роботи**

Описати алгебраїчний тип даних через незмінні dataclass-вузли та TypeAlias. Застосувати Literal, Protocol і структурний pattern matching для чистої обробки журналу подій; перевірити модель тестами та mypy --strict.

**Теоретичні відомості**

ADT як сума варіантів задає обмежений набір форм даних. Union об’єднує класи, Literal звужує допустимі значення, а Protocol описує структурний контракт. Match/case розбирає форму вузла й дістає його поля.

Frozen dataclass забороняє звичайне переприсвоєння полів. Tuple і frozenset зберігають незмінність вкладених колекцій. Assert_never у фінальній гілці допомагає mypy контролювати вичерпність розбору при розширенні типу.

**Хід роботи**

Для варіанта 14 я реалізував події Login, Logout, FileOpened та ErrorEvent. Вони мають timestamp, а ErrorEvent — рівень із Literal: warning, error або critical. Event об’єднує чотири класи через TypeAlias. Timestamp перевіряється на скінченність і невід’ємність.

Protocol HasTimestamp задає властивість timestamp тільки для читання. Події структурно сумісні з ним без успадкування. У тесті той самий контракт виконує сторонній dataclass Stamp. EventTree рекурсивно описує окрему подію або tuple вкладених журналів.

У evaluator.py я створив чистий reducer(state, event) із match для всіх чотирьох варіантів і вкладеним match рівнів помилки. Він повертає новий Stats через replace: лічильники, frozenset активних користувачів, tuple шляхів і максимальний timestamp. Вхідний стан не змінюється.

Flatten рекурсивно обходить вкладені tuple зліва направо, а process згортає події через reduce. Порядок журналу визначає переходи активних користувачів; timestamp не пересортовує записи. Друга операція — render_event із match та текстове представлення журналу й статистики.

Main.py демонструє вкладений журнал із 7 подій: 2 входи, 1 вихід, 1 відкриття файлу та 3 повідомлення різних рівнів. Активним залишається Sviatoslav. Початковий стан має 0 подій; повторна обробка з порожнього стану дає однаковий результат.

Тести охоплюють усі варіанти ADT, вкладені та порожні журнали, незмінність, Protocol, граничні значення й рендер.

lab08 / model.py

```python
"""ADT подій системи, незмінний стан та структурний контракт timestamp."""

from __future__ import annotations

from dataclasses import dataclass
from math import isfinite
from typing import Literal, Protocol, TypeAlias

ErrorLevel: TypeAlias = Literal["warning", "error", "critical"]


class HasTimestamp(Protocol):
    @property
    def timestamp(self) -> float: ...


def timestamp_of(event: HasTimestamp) -> float:
    return event.timestamp


def check_timestamp(value: float) -> None:
    if not isfinite(value) or value < 0:
        raise ValueError("timestamp має бути скінченним невід'ємним числом")


@dataclass(frozen=True, slots=True)
class Login:
    timestamp: float
    user: str

    def __post_init__(self) -> None:
```

Рис. 1. Файл model.py, рядки 1–31

lab08 / model.py

```python
        check_timestamp(self.timestamp)


@dataclass(frozen=True, slots=True)
class Logout:
    timestamp: float
    user: str

    def __post_init__(self) -> None:
        check_timestamp(self.timestamp)


@dataclass(frozen=True, slots=True)
class FileOpened:
    timestamp: float
    user: str
    path: str

    def __post_init__(self) -> None:
        check_timestamp(self.timestamp)


@dataclass(frozen=True, slots=True)
class ErrorEvent:
    timestamp: float
    level: ErrorLevel
    message: str

    def __post_init__(self) -> None:
        check_timestamp(self.timestamp)
        if self.level not in {"warning", "error", "critical"}:
```

Рис. 2. Файл model.py, рядки 32–62

lab08 / model.py

```
63              raise ValueError("Невідомий рівень помилки")
64
65
66  Event: TypeAlias = Login | Logout | FileOpened | ErrorEvent
67  EventTree: TypeAlias = Event | tuple["EventTree", ...]
68
69
70  @dataclass(frozen=True, slots=True)
71  class Stats:
72      logins: int = 0
73      logouts: int = 0
74      files_opened: int = 0
75      warnings: int = 0
76      errors: int = 0
77      criticals: int = 0
78      active_users: frozenset[str] = frozenset()
79      paths: tuple[str, ...] = ()
80      last_timestamp: float | None = None
81
82      @property
83      def total(self) -> int:
84          return (
85              self.logins
86              + self.logouts
87              + self.files_opened
88              + self.warnings
89              + self.errors
90              + self.criticals
91          )
```

Рис. 3. Файл model.py, рядки 63–91

lab08 / evaluator.py

```
1   """Чиста інтерпретація ADT через вичерпний match/case."""
2
3   from collections.abc import Iterator
4   from dataclasses import replace
5   from functools import reduce
6   from typing import assert_never
7
8   from model import (
9       ErrorEvent,
10      Event,
11      EventTree,
12      FileOpened,
13      Login,
14      Logout,
15      Stats,
16      timestamp_of,
17  )
18
19
20  def reducer(state: Stats, event: Event) -> Stats:
21      timestamp = timestamp_of(event)
22      latest = (
23          timestamp
24          if state.last_timestamp is None
25          else max(timestamp, state.last_timestamp)
26      )
27      base = replace(state, last_timestamp=latest)
28      match event:
29          case Login(user=user):
30              return replace(
31                  base, logins=base.logins + 1, active_users=base.active_users | {user}
32              )
33          case Logout(user=user):
34              return replace(
```

Рис. 4. Файл evaluator.py, рядки 1–34

lab08 / evaluator.py

```
                base, logouts=base.logouts + 1, active_users=base.active_users - {user}
            )
        case FileOpened(path=path):
            return replace(
                base, files_opened=base.files_opened + 1, paths=base.paths + (path,)
            )
        case ErrorEvent(level=level):
            match level:
                case "warning":
                    return replace(base, warnings=base.warnings + 1)
                case "error":
                    return replace(base, errors=base.errors + 1)
                case "critical":
                    return replace(base, criticals=base.criticals + 1)
                case _:
                    assert_never(level)
        case _:
            assert_never(event)


def flatten(journal: EventTree) -> Iterator[Event]:
    match journal:
        case tuple(children):
            for child in children:
                yield from flatten(child)
        case Login() | Logout() | FileOpened() | ErrorEvent():
            yield journal
        case _:
            assert_never(journal)


def process(journal: EventTree, initial: Stats | None = None) -> Stats:
    state = Stats() if initial is None else initial
    return reduce(reducer, flatten(journal), state)
```

Рис. 5. Файл evaluator.py, рядки 35–68

lab08 / pretty.py

```
"""Чисте текстове представлення подій і підсумків."""

from typing import assert_never

from evaluator import flatten
from model import ErrorEvent, Event, EventTree, FileOpened, Login, Logout, Stats


def render_event(event: Event) -> str:
    match event:
        case Login(timestamp=t, user=user):
            return f"t={t:g} LOGIN {user}"
        case Logout(timestamp=t, user=user):
            return f"t={t:g} LOGOUT {user}"
        case FileOpened(timestamp=t, user=user, path=path):
            return f"t={t:g} FILE {user}: {path}"
        case ErrorEvent(timestamp=t, level=level, message=message):
            return f"t={t:g} {level.upper()}: {message}"
        case _:
            assert_never(event)


def render_journal(journal: EventTree) -> str:
    return "\n".join(map(render_event, flatten(journal)))


def render_stats(state: Stats) -> str:
    return (
        f"Усього подій: {state.total}\n"
        f"Login: {state.logins}; Logout: {state.logouts}; файлів: {state.files_opened}\n"
        f"warning: {state.warnings}; error: {state.errors}; critical: {state.criticals}\n"
        f"Активні користувачі: {', '.join(sorted(state.active_users)) or 'немає'}\n"
        f"Максимальний timestamp: {state.last_timestamp}"
    )
```

Рис. 6. Файл pretty.py, рядки 1–34

lab08 / main.py

```python
"""Демонстрація варіанта 14, без реального входу чи читання файлів."""

from evaluator import process
from model import ErrorEvent, EventTree, FileOpened, Login, Logout, Stats
from pretty import render_journal, render_stats


def main() -> None:
    journal: EventTree = (
        Login(1, "Sviatoslav"),
        (
            FileOpened(2, "Sviatoslav", "report.odt"),
            ErrorEvent(3, "warning", "slow task"),
        ),
        (Login(4, "Guest"), (ErrorEvent(5, "error", "timeout"),)),
        Logout(6, "Guest"),
        ErrorEvent(7, "critical", "service unavailable"),
    )
    original = Stats()
    result = process(journal, original)
    print("=== ПОДІЇ СИСТЕМИ | ВАРІАНТ 14 ===")
    print(render_journal(journal))
    print("--- Статистика ---")
    print(render_stats(result))
    print("Початковий стан, подій:", original.total)
    print("Повторний запуск дає той самий результат:", process(journal) == result)


if __name__ == "__main__":
    main()
```

Рис. 7. Файл main.py, рядки 1–30

lab08 / tests/test_events.py

```python
from dataclasses import FrozenInstanceError, dataclass
from typing import cast

import pytest

from evaluator import flatten, process, reducer
from model import (
    ErrorEvent,
    ErrorLevel,
    Event,
    EventTree,
    FileOpened,
    Login,
    Logout,
    Stats,
    timestamp_of,
)
from pretty import render_event, render_journal, render_stats


@pytest.mark.parametrize(
    "event,field",
    [
        (Login(1, "A"), "logins"),
        (Logout(1, "A"), "logouts"),
        (FileOpened(1, "A", "a.txt"), "files_opened"),
        (ErrorEvent(1, "warning", "w"), "warnings"),
        (ErrorEvent(1, "error", "e"), "errors"),
        (ErrorEvent(1, "critical", "c"), "criticals"),
    ],
)
```

Рис. 8. Файл tests/test_events.py, рядки 1–31

lab08 / tests/test_events.py

```
32  def test_each_variant(event: Event, field: str) -> None:
33      original = Stats()
34      result = reducer(original, event)
35      assert getattr(result, field) == 1 and result.total == 1
36      assert original.total == 0 and result is not original
37      assert result.last_timestamp == 1
38
39
40  def test_nested_journal_and_sessions() -> None:
41      journal: EventTree = (
42          Login(1, "A"),
43          (Login(2, "B"), (FileOpened(3, "A", "x"),)),
44          Logout(4, "B"),
45          ErrorEvent(5, "warning", "w"),
46      )
47      initial = Stats()
48      result = process(journal, initial)
49      assert result.total == 5 and result.active_users == frozenset({"A"})
50      assert result.paths == ("x",) and result.last_timestamp == 5
51      assert initial == Stats()
52      assert process(journal) == process(journal)
53      assert len(tuple(flatten(journal))) == 5
54
55
56  def test_empty_single_and_incremental() -> None:
57      assert process(()) == Stats()
58      assert render_journal(()) == ""
59      initial = process(Login(1, "A"))
60      result = process((Logout(2, "A"),), initial)
61      assert result.total == 2 and not result.active_users
62      assert initial.active_users == frozenset({"A"})
```

Рис. 9. Файл tests/test_events.py, рядки 32–62

lab08 / tests/test_events.py

```
63
64
65  def test_duplicates_and_out_of_order() -> None:
66      result = process((Login(10, "A"), Login(2, "A"), Logout(3, "unknown")))
67      assert result.logins == 2 and result.logouts == 1
68      assert result.active_users == frozenset({"A"})
69      assert result.last_timestamp == 10
70
71
72  def test_protocol_without_inheritance() -> None:
73      @dataclass(frozen=True)
74      class Stamp:
75          timestamp: float
76
77      assert timestamp_of(Stamp(12.0)) == 12.0
78      assert timestamp_of(Login(1, "A")) == 1
79
80
81  def test_frozen_nodes_and_state() -> None:
82      node = Login(1, "A")
83      state = process(node)
84      with pytest.raises(FrozenInstanceError):
85          node.user = "B"  # type: ignore[misc]
86      with pytest.raises(FrozenInstanceError):
87          state.logins = 99  # type: ignore[misc]
88      assert state.active_users == frozenset({"A"})
89
90
91  @pytest.mark.parametrize("timestamp", [-1, float("inf"), float("nan")])
92  def test_invalid_timestamp(timestamp: float) -> None:
93      for build in (
```

Рис. 10. Файл tests/test_events.py, рядки 63–93

lab08 / tests/test_events.py

```
 94          lambda: Login(timestamp, "A"),
 95          lambda: Logout(timestamp, "A"),
 96          lambda: FileOpened(timestamp, "A", "x"),
 97          lambda: ErrorEvent(timestamp, "error", "x"),
 98      ):
 99          with pytest.raises(ValueError):
100              build()
101
102
103  def test_invalid_level_and_unknown_node() -> None:
104      with pytest.raises(ValueError):
105          ErrorEvent(1, cast(ErrorLevel, "fatal"), "bad")
106      with pytest.raises(AssertionError):
107          render_event(cast(Event, object()))
108      with pytest.raises(AssertionError):
109          tuple(flatten(cast(EventTree, "invalid")))
110
111
112  @pytest.mark.parametrize(
113      "event,expected",
114      [
115          (Login(1, "A"), "t=1 LOGIN A"),
116          (Logout(2, "A"), "t=2 LOGOUT A"),
117          (FileOpened(3, "A", "x"), "t=3 FILE A: x"),
118          (ErrorEvent(4, "error", "bad"), "t=4 ERROR: bad"),
119      ],
120  )
121  def test_render(event: Event, expected: str) -> None:
122      assert render_event(event) == expected
123      assert render_journal((event,)) == expected
124      assert "Усього подій: 1" in render_stats(process(event))
```

Рис. 11. Файл tests/test_events.py, рядки 94–124

## Запуск і результати перевірки

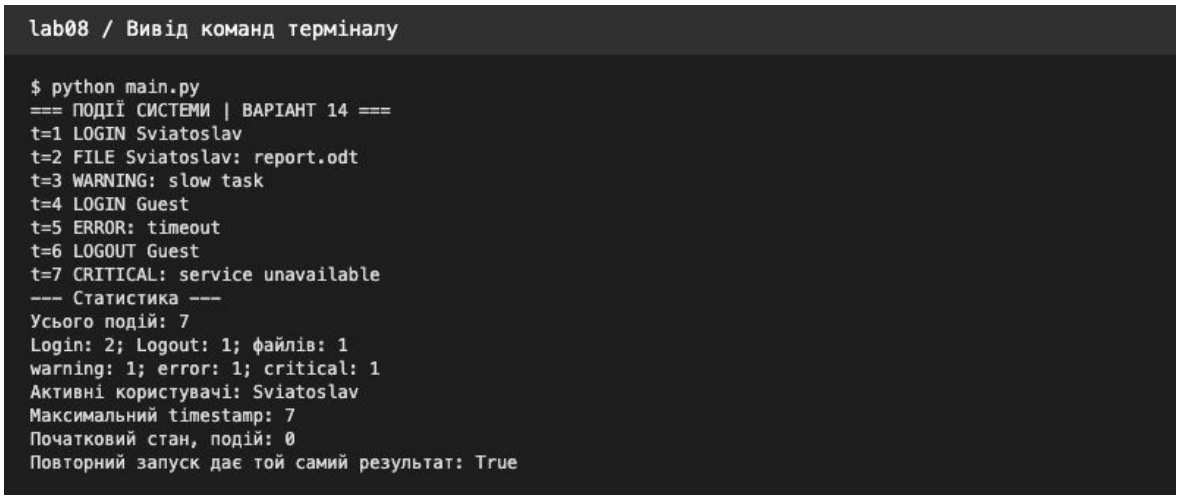



Рис. 12. Запуск демонстрації

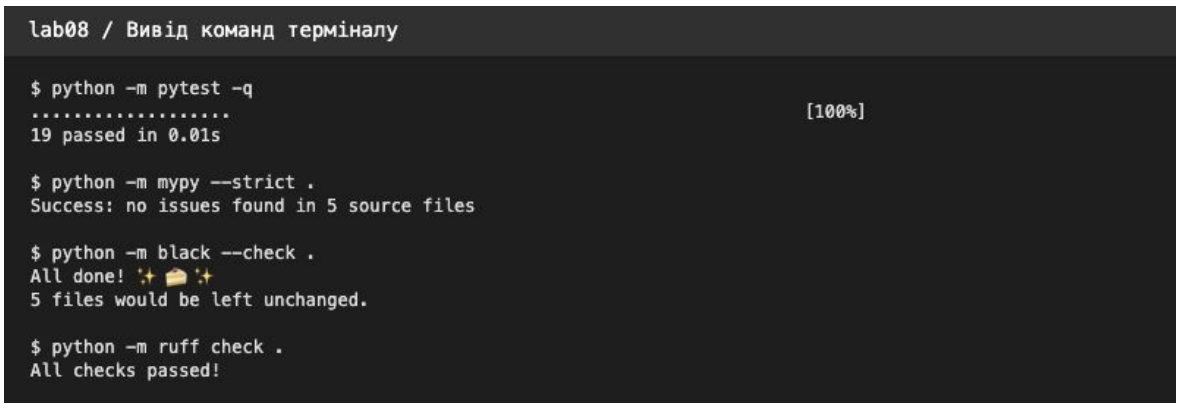



Рис. 13. Тести та перевірки якості

## Висновок

Я створив ADT подій системи з незмінними вузлами, Literal для рівнів помилки та структурним Protocol. Реалізував чистий reducer, обробку вкладених журналів і текстове представлення через match/case. Початкові події та стан не змінюються.

Усі 19 тестів пройшли; mypy --strict, black і ruff помилок не виявили. Для демонстраційного журналу статистика містить 7 подій і одного активного користувача. Явні варіанти ADT та assert_never допомагають підтримувати вичерпність обробки.

Роботу виконано за наданим Lab_08.docx, індивідуальний варіант 14.